



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

Name: Saurav paudel

PRN: 25070122174

Branch: CSE (2025–29)

Semester: 3

Subject: Programming with Java

ASSIGNMENT No :2

TITLE :Develop a Java application demonstrating the use of default, parameterized, and copy constructors for object initialization and data management.

Assignment 2: Methods and Static Members

Problem Statement

Develop a Java application demonstrating the use of default, parameterized, and copy constructors for object initialization and data management.

Learning Objectives

To understand and implement default, parameterized, and copy constructors for object initialization.

Theory

Java supports default constructors provided by the compiler, parameterized constructors for initializing objects with user-defined values, and copy constructors (implemented manually) for creating duplicate objects. Constructors improve code readability and simplify object initialization while supporting object-oriented programming principles.

Constructors are special member functions in Java that are automatically invoked when an object is created. They have the same name as the class and no return type. Java supports several types of constructors:

- **Default constructor** — provided automatically by the compiler if no constructor is defined, or written explicitly to initialize objects with standard/default values.

- **Parameterized constructor** — accepts arguments so that objects can be initialized with user-defined values at the time of creation.
- **Copy constructor** — not built into Java like C++, but can be implemented manually by passing an object of the same class as a parameter and copying its field values into the new object.

Constructors improve code readability, reduce the need for separate setter calls, and simplify object initialization, which is a core part of object-oriented programming.

for object initialization and data duplication.

Exercise

1. Create a **Student** class using default and parameterized constructors to initialize student name and roll number.

Source code:

```
Student.java > Student > Student(int, String)
1  public class Student {
2
3      int id;
4      String name;
5
6      // Default Constructor
7      Student() {
8          id = 154;
9          name = "Saurav Paudel";
10     }
11
12     // Parameterized Constructor
13     Student(int i, String n) {
14         id = i;
15         name = n;
16     }
17
18     // Copy Constructor
19     Student(Student s) {
20         id = s.id;
21         name = s.name;
22     }
23
24     // Display Method
25     void display() {
26         System.out.println("ID : " + id);
27         System.out.println("Name : " + name);
28         System.out.println();
29     }
30
31     Run | Debug
32     public static void main(String[] args) {
33
34         // Default Constructor
35         Student s1 = new Student();
36
37         // Parameterized Constructor
38         Student s2 = new Student(i: 5652555, n: "Saurav Paudel");
39     }
```

```
System.out.println("Name : " + name);
System.out.println();
}

Run | Debug
public static void main(String[] args) {

    // Default Constructor
    Student s1 = new Student();

    // Parameterized Constructor
    Student s2 = new Student(i: 5652555, n: "Saurav Paudel");

    // Copy Constructor
    Student s3 = new Student(s2);

    System.out.println(x: "Default Constructor:");
    s1.display();

    System.out.println(x: "Parameterized Constructor:");
    s2.display();

    System.out.println(x: "Copy Constructor:");
    s3.display();
}
```

Output:

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sitap>cd C:\Users\sitap\OneDrive\Desktop\java\colleague ass
C:\Users\sitap\OneDrive\Desktop\java\colleague ass>javac Student.java
C:\Users\sitap\OneDrive\Desktop\java\colleague ass>java Student
Default Constructor:
ID : 154
Name : Saurav Paudel

Parameterized Constructor:
ID : 5652555
Name : Saurav Paudel

Copy Constructor:
ID : 5652555
Name : Saurav Paudel

C:\Users\sitap\OneDrive\Desktop\java\colleague ass>
```

2. Develop a **Mobile Phone Inventory System** using different constructors to initialize mobile details and create duplicate object records.

Source code:

```
J EmployeePayrollSystem.java > EmployeePayrollSystem
1  import java.util.Scanner;
2
3  public class EmployeePayrollSystem {
4
5      Run | Debug
6      public static void main(String[] args) {
7
8          Scanner sc = new Scanner(System.in);
9
10         System.out.print(s: "Enter Employee ID: ");
11         Integer empId = Integer.valueOf(sc.nextLine());
12
13         System.out.print(s: "Enter Basic Salary: ");
14         Double basicSalary = Double.valueOf(sc.nextLine());
15
16         System.out.print(s: "Enter Bonus: ");
17         Double bonus = Double.valueOf(sc.nextLine());
18
19         if (basicSalary < 0 || bonus < 0) {
20             System.out.println(x: "Invalid salary values entered.");
21         } else {
22             double netSalary = basicSalary.doubleValue() + bonus.doubleValue();
23
24             System.out.println("Employee ID: " + empId);
25             System.out.println("Net Salary: Rs. " + netSalary);
26         }
27
28         sc.close();
29     }
```

Output:

```
C:\Users\sitap\OneDrive\Desktop\java\colleage ass>javac EmployeePayrollSystem.java
C:\Users\sitap\OneDrive\Desktop\java\colleage ass>java EmployeePayrollSystem
Enter Employee ID: 2507
Enter Basic Salary: 8585
Enter Bonus: 33
Employee ID: 2507
Net Salary: Rs. 8618.0
C:\Users\sitap\OneDrive\Desktop\java\colleage ass>|
```

Conclusion

This program demonstrates how Java constructors are used for object initialization in different scenarios. The default constructor assigns fixed/standard values when no data is provided, the parameterized constructor allows initializing an object with specific values at creation time, and the copy constructor (implemented manually) duplicates the data of an existing object into a new one. This shows how constructors support flexible and efficient object creation, which is essential in real-world data management systems.

